UNIVESIDAD TECNOLOGICA NACIONAL

FACULTAD REGIONAL MENDOZA

Carrera: tecnicatura universitaria en programación

Materia: Programación II

FOOD STORE

Sistema de gestión de pedidos de comidas

Integrantes:

Fernando Pimentel

Fabricio Pimentel

Luis Pimentel

Fecha de entrega: 9/11/2026

INDICE

1. Introducción y alcance del proyecto 1

2. Marco Teórico 1
   2.1. Clase abstracta 1
   2.2. Interfaz 2
   2.3. Herencia 2
   2.4. Polimorfismo 3
   2.5. Encapsulamiento 4
   2.6. Composición/Agregación 4
   2.7. Excepciones 5
   2.8. Colecciones 6
   2.9. Type hints 7

3. Decisiones técnicas y arquitectura 8

4. Dificultades y Conclusiones 14

5. Bibliografía y Webgrafía 15

6. Enlace video explicativo 15

7. Enlace repositorio GitHub 15

1. INTRODUCCIÓN:

El presente informe detalla el diseño, desarrollo e implementación del sistema de consola Food Store, un software orientado a la gestión integral de un negocio de comidas. El desarrollo se ha guiado bajo el paradigma de la Programación Orientada a Objetos (POO) utilizando el lenguaje de programación Python.

Alcance del Proyecto

- **Interfaz de Consola:** Toda la interacción del usuario ocurre mediante menús de opciones en texto.
- **Control de Acceso:** El sistema incorpora un inicio de sesión por roles (ADMIN y USUARIO), restringiendo las operaciones de alta, baja y modificación al personal autorizado.
- **Persistencia en Memoria:** Los datos del dominio se administran utilizando colecciones en memoria durante el tiempo de ejecución.
- **Robustez y Tipado:** Se aplican validaciones estrictas, manejo de excepciones personalizadas para el control de errores de negocio y tipado estricto (*type hints*) en todo el código fuente.

2. MARCO TEORICO:

2.1. **Clase Abstracta:**

En Python, una clase abstracta es una clase que no puede instanciarse por sí misma y está diseñada para servir de modelo para otras clases. Las clases abstractas nos permiten definir métodos que deben ser implementados por las subclases, lo que garantiza una interfaz consistente a la vez que permite que las subclases proporcionen implementaciones específicas. Se crea utilizando el módulo abc (Abstract Base Classes), heredando de la clase ABC y marcando los métodos obligatorios con el decorador @abstractmethod.

Las características principales son:

- Instanciación prohibida: Intentar crear un objeto de una clase abstracta lanza un error TypeError.
- Implementación obligatoria: Las clases que heredan deben proporcionar el cuerpo de todos los métodos abstractos para poder ser instanciadas.
- Métodos concretos: Pueden incluir métodos con implementación completa que las subclases heredan automáticamente.
- Interfaces simuladas: Al declarar todos los métodos como abstractos, se comportan como interfaces formales.

Ejemplo básico de definición y uso:

```
from  abc  import  ABC , abstractmethod
Clase  Animal ( ABC ):
    @abstractmethod
    def  sonido ( yo mismo ):
        aprobar

clase  Perro ( Animal ):
    def  sonido ( yo mismo ):
        devolver  "Ladrido"

perro  =  Perro ()
imprimir ( perro . sonido ())
```

2.2. Interfaz

En Python no tiene una palabra clave específica para esto. En su lugar, implementa las interfaces de tres formas:

- **Interfaces informales:** Se crean mediante una clase base donde los métodos solo contienen pass o levantan una excepción (NotImplementedError), confiando en que el programador los sobrescriba en las subclases.
- **Interfaces formales (ABC):** Se usa el módulo nativo abc (*Abstract Base Class*) con el decorador @abstractmethod para obligar a las clases hijas a implementar ciertos métodos:

```
from abc import ABC, abstractmethod

class LectorArchivos(ABC):
    @abstractmethod
    .leer(self):
        pass
```

- **Protocolos (Tipado estructural / *Duck Typing*):** Mediante typing.Protocol, Python permite verificar de manera estática (con herramientas como mypy) si un objeto cumple con una "forma" o conjunto de métodos, sin necesidad de que exista una herencia formal.

2.3. Herencia

La herencia es un concepto fundamental en la programación orientada a objetos (POO) que permite que una clase (llamada clase hija o derivada) herede atributos y métodos de otra clase (llamada clase padre o base).

Aquí se crea una clase padre, Animal, que tiene un método info(). Luego se crea una clase hija, Perro, que hereda de Animal y añade un comportamiento adicional.

```
clase  Animal :
    def  __init__ ( self , nombre ):
        nombre .

    def  info ( self ):
        imprimir ( "Nombre del animal:" , self . nombre )

clase  Perro ( Animal ):
    def  sonido ( yo mismo ):
        imprimir ( self.name , " ladra " )

d  =  Perro ( "Amigo" )
# Método heredado
d.info ( )
d . sonido ()
```

Salida

```
Nombre del animal: Buddy
Buddy ladra
```

2.4. Polimorfismo

El polimorfismo es la capacidad de tratar objetos de distintas subclases de manera uniforme: le pedís a cada uno la misma operación y cada uno responde a su modo. Su poder aparece cuando recorres una colección heterogénea —una lista con objetos de varios tipos— invocando el mismo método sobre todos, sin preguntar de qué tipo es cada uno.

Ejemplo practico de polimorfismo en Python:

```python
class Perro:
    def hablar(self):
        return "Guau!"

class Gato:
    def hablar(self):
        return "Miau!"

class Vaca:
    def hablar(self):
        return "Muuu!"

# Función que utiliza polimorfismo
def hacer_hablar(animal):
    print(animal.hablar())

# Creamos los objetos
p = Perro()
g = Gato()
v = Vaca()

# Llamamos a la misma función con diferentes objetos
hacer_hablar(p)  # Salida: Guau!
hacer_hablar(g)  # Salida: Miau!
hacer_hablar(v)  # Salida: Muuu!
```

2.5. Encapsulamiento

La encapsulación es uno de los conceptos fundamentales de la Programación Orientada a Objetos (POO).

- La idea de la encapsulación es vincular los miembros de datos y los métodos en una sola unidad.
- Ayuda a mejorar la mantenibilidad, la legibilidad y la usabilidad, ya que no necesitamos pasar explícitamente miembros de datos a los métodos de los miembros.
- Ayuda a mantener la integridad de los datos al permitir la validación y el control sobre los valores asignados a las variables.
- Esto ayuda a lograr la abstracción . Una clase puede ocultar la parte de implementación y revelar solo las funcionalidades requeridas por otras clases, lo que permite realizar cambios posteriores en las representaciones o implementaciones sin afectar el código que utiliza esta clase.

Este ejemplo muestra la encapsulación al mantener la variable __salary privada dentro de la clase Employee. No se puede acceder a ella directamente desde fuera de la clase.

```
clase  Empleado :
    def  __init__ ( self , nombre , salario ):
        self.nombre = nombre  # atributo público
        self .__ salario  =  salario     # atributo privado

emp  =  Empleado ( "Fedrick" , 50000 )
imprimir ( emp.nombre )
imprimir ( empleado . __salario )
```

2.6. Composición/Agregación

Composición:

Como la parte no vive sin el todo, el todo la crea. En el práctico esto se traduce en una regla clara: el constructor del todo crea la parte adentro, con self._parte = Parte(...). La parte nace y muere con el todo.

El guion bajo no es un adorno: en Python marca que ese atributo es interno, algo que nadie de afuera debería reemplazar —justo lo que dice el rombo del diagrama—. Si la parte igual necesita leerse desde afuera, se expone con una @property de solo lectura, como viste en la unidad de encapsulamiento. La misma convención vale para la parte de una agregación; en cambio, una referencia de asociación (el autor de un libro, el titular de un pasaporte) queda como atributo público, porque ahí no hay ninguna parte que proteger.

```python
class Foto:
    def __init__(self, imagen, formato):
        self.imagen = imagen
        self.formato = formato

class Pasaporte:
    def __init__(self, numero, fecha_emision):
        self.numero = numero
        self.fecha_emision = fecha_emision
        self._foto = Foto("rostro.jpg", "JPG")  # ◆ se CREA adentro

    @property
    def foto(self):          # se lee, pero no se reemplaza desde afuera
        return self._foto

if __name__ == "__main__":
    p = Pasaporte("AR123", "2026-01-01")
    print(p.foto.formato)     # 'JPG' — la foto nació con el pasaporte
    # nadie construyó la Foto desde afuera: no existe sin este pasaporte
```

Agregación:

Como la parte puede existir sola, llega ya construida desde afuera. La regla del práctico: el constructor del todo recibe la parte como parámetro y la guarda; no la crea. La misma batería podría ir a otro celular.

```python
class Bateria:
    def __init__(self, modelo, capacidad):
        self.modelo = modelo
        self.capacidad = capacidad

class Celular:
    def __init__(self, imei, bateria):     # ◇ la RECIBE por parámetro
        self.imei = imei
        self._bateria = bateria

    @property
    def bateria(self):
        return self._bateria

if __name__ == "__main__":
    b = Bateria("Li-3000", 3000)      # la batería existe por su cuenta
    c = Celular("356938", b)          # y se la pasamos al celular
    print(c.bateria.capacidad)        # 3000
```

Simbología UML

- Composición: rombo lleno (◆) del lado del todo.
- Agregación: rombo hueco (◇) del lado del todo.

El rombo siempre va del lado del todo (el que contiene), no del lado de la parte.

2.7. Excepciones

Una excepción es un objeto que representa un error. Cuando algo sale mal, Python «lanza» una excepción: interrumpe la ejecución normal y busca a alguien que la maneje. Si nadie la maneja, el programa termina y muestra el traceback. La última línea del traceback te dice el tipo exacto del error y el mensaje. Ese nombre es la información que necesitás:

```
Traceback (most recent call last):
  File "prueba.py", line 3, in <module>
    resultado = 10 / 0
ZeroDivisionError: division by zero
```

Las categorías de excepciones más comunes incluyen:

Errores de tipo y valor: TypeError (operación en tipo incorrecto) y ValueError (valor inapropiado para el tipo correcto).

Errores de nombre: NameError (nombre no definido) y UnboundLocalError.

Errores de búsqueda: IndexError (fuera de rango en secuencias) y KeyError (clave no encontrada en diccionarios).

Errores aritméticos: ZeroDivisionError, OverflowError y FloatingPointError.

Errores de sistema operativo: Agrupadas bajo OSError, incluyen FileNotFoundError, PermissionError y errores de conexión como ConnectionRefusedError.

Errores de sintaxis y carga: SyntaxError, IndentationError y ImportError (con su subclase ModuleNotFoundError).

El manejo se realiza mediante bloques try, except, else y finally, donde un except puede capturar múltiples excepciones o heredar de clases base para manejar grupos de errores relacionados.

```python
def division_segura(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print("No se puede dividir por cero.")
        return None

if __name__ == "__main__":
    print(division_segura(10, 2))    # 5.0
    print(division_segura(10, 0))    # aviso + None
```

2.8. Colecciones

Una colección permite agrupar varios objetos bajo un mismo nombre. Por ejemplo, si necesitamos almacenar en nuestro programa los nombres de los alumnos de un curso de programación, será más conveniente ubicarlos a todos dentro de una misma colección de nombre alumnos, en lugar de crear los objetos alumno1, alumno2, etc.

En Python existen tres colecciones básicas, a saber: las listas, las tuplas y los diccionarios.

- Listas:

  Una lista es un conjunto ordenado de objetos. Por objetos entendemos cualquiera de los tipos de dato ya mencionados, incluso otras listas.

```
>>> lenguajes = ["Python", "Java", "C", "C++"]
```

- Tuplas:
  Las tuplas, al igual que las listas, son colecciones ordenadas. No obstante, a diferencia de éstas, son inmutables. Es decir, una vez asignados los elementos, no pueden ser alterados.

```
# Indicamos los elementos entre paréntesis.
>>> lenguajes = ("Python", "Java", "C", "C++")
>>> lenguajes[0]
'Python'
>>> lenguajes[-1]
'C++'
```

- Diccionarios:
  Los diccionarios, a diferencia de las listas y las tuplas, son colecciones no ordenadas de objetos. Además, sus elementos tienen una particularidad: siempre conforman un par clave-valor.

  Para crear un diccionario, indicamos los pares clave-valor separados por comas y estos, a su vez, separados por dos puntos.

```
>>> d = {"Python": 1991, "C": 1972, "Java": 1996}
```

2.9. Type hints

Las type hints (o anotaciones de tipo) son una sintaxis introducida en Python 3.5 que permite especificar los tipos de datos esperados para variables, argumentos de funciones y valores de retorno. Su propósito principal es mejorar la legibilidad, facilitar el autocompletado en IDEs y permitir la detección temprana de errores mediante herramientas de análisis estático como mypy o pyright.

Sintaxis Básica

La anotación se realiza utilizando dos puntos (:) para parámetros/variables y una flecha (->) para el retorno:

```
# Variable
edad: int = 30

# Función
def saludar(nombre: str) -> str:
    return f"Hola, {nombre}"
```

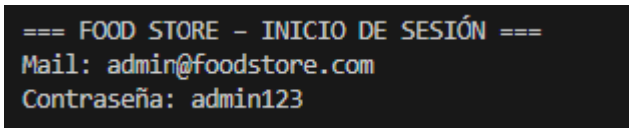
3. DECISINES TECNICAS Y ARQUITECTURA

Para cumplir con los objetivos de diseño modular, el sistema **Food Store** se estructuró bajo una **arquitectura en capas estrictas**, separando responsabilidades y asegurando que las dependencias fluyan en un único sentido (Consola → Servicios → Entidades):

- **Capa de Entidades (Modelo):** Contiene las clases del dominio, aplicando encapsulamiento, validaciones en propiedades y herencia de una clase abstracta Base.
- **Capa de Servicios (Lógica de Negocio):** Aloja las colecciones en memoria y procesa las reglas de negocio complejas (como unicidad de correos, control de stock y transiciones de estados) de forma totalmente desacoplada de la interfaz de usuario.
- **Capa de Consola (Interfaz):** Maneja la interacción mediante menús basados en polimorfismo (MenuAdministrador y MenuCliente), capturando excepciones de negocio para informar al usuario sin cerrar la aplicación.

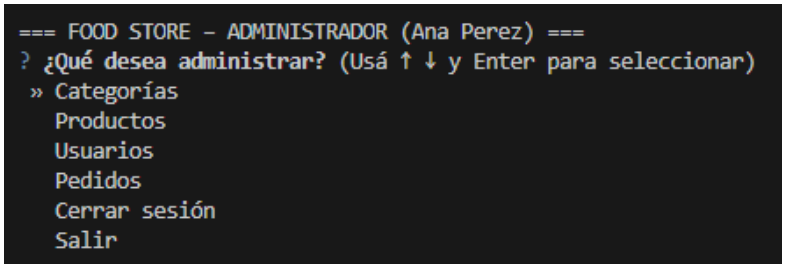
Capturas de pantalla del sistema funcionando

INGRESO AL SISTEMA



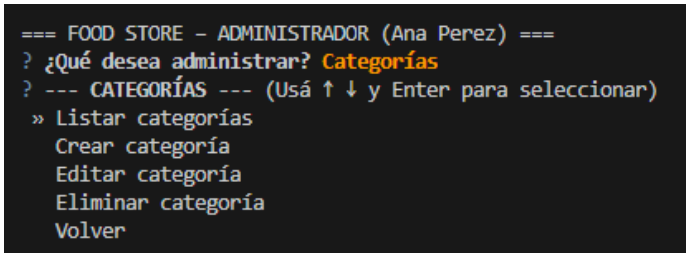
Descripción: se muestra el menú principal del sistema, donde el usuario o el administrador deben colocar su email y contraseña para poder ingresar al sistema. En este ejemplo se ingresó como administrador.

MENÚ ADMINISTRADOR:



Descripción: Menú principal del rol Administrador. Permite acceder a los submenús de gestión de categorías, productos, usuarios y pedidos

CATEGORIAS:



Descripción: Al seleccionar la opción categorías el sistema les mostrara un submenú donde podrán: listar categorías, crear categoría, editar categoría, eliminar categoría, y la opción de volver que lo llevara al menú principal.

Listar categorías:

```
? --- CATEGORÍAS --- Listar categorías
ID: 2 | Hamburguesas | Hamburguesas y combos
```

Descripción: Listado de categorías activas en el sistema. Muestra el ID, el nombre y la descripción de cada una de las categorías registradas.

Crear categoría:

```
? --- CATEGORÍAS --- Crear categoría
Nombre: bebidas
Descripción: Bebidas frías, gaseosas, jugos naturales y opciones sin alcohol para acompañar tus comidas.
Categoría creada. ID: 4
```

Descripción: Proceso de creación de una nueva categoría ("Bebidas"). El sistema valida dinámicamente que los campos ingresados cumplan con las reglas de negocio (campos no vacíos y unicidad del nombre).

Editar categoría:

```
? --- CATEGORÍAS --- Editar categoría
ID: 2 | Hamburguesas | Hamburguesas y combos
ID: 4 | bebidas | Bebidas frías, gaseosas, jugos naturales y opciones sin alcohol para acompañar tus comidas.
ID a editar: 4
Nombre [bebidas]: Bebidas
Descripción [Bebidas frías, gaseosas, jugos naturales y opciones sin alcohol para acompañar tus comidas.]: gaseosas, jugos, aguas, sodas.
Categoría actualizada.
```

Descripción: Proceso de edición de una categoría existente. Ingresando el identificador único (ID: 4), el sistema permite modificar sus atributos (nombre y descripción), actualizando correctamente los datos en la memoria.

Eliminar categoría:

```
? --- CATEGORÍAS --- Eliminar categoría
ID: 2 | Hamburguesas | Hamburguesas y combos
ID: 4 | Bebidas | gaseosas, jugos, aguas, sodas.
ID a eliminar: 4
? ¿Confirma la eliminación? Sí
Categoría eliminada.
```

Descripción: Proceso de eliminación lógica de una categoría. El sistema lista los registros disponibles, solicita el identificador único del elemento a dar de baja (ID: 4) y requiere una confirmación previa por parte del administrador antes de aplicar la modificación en memoria.

PRODUCTOS:

```
? --- PRODUCTOS --- (Usá ↑ ↓ y Enter para seleccionar)
 » Listar productos
   Crear producto
   Editar producto
   Eliminar producto
   Volver
```

Descripción: Al seleccionar la opción productos, el sistema les mostrara un submenú donde podrán: listar productos, crear producto, editar producto, eliminar producto, y la opción de volver que lo llevara al menú principal.

Listar producto:

```
? ¿Qué desea administrar? Productos
? --- PRODUCTOS --- Listar productos
ID: 3 | Hamburguesa completa | $4500.00 | Stock: 10 | Categoría: Hamburguesas
```

Descripción: Listado de productos registrados en el sistema. Se visualiza la información del producto (ID único de producto, nombre, precio, stock y categoría asociada).

Crear producto:

```
? --- PRODUCTOS --- Crear producto
ID: 2 | Hamburguesas | Hamburguesas y combos
ID categoría: 2
Nombre: hambusguesa mediana
Precio: 3550
Descripción: Medallón de carne vacuna, huevo a la plancha, lechuga y tomate en pan brioche.
Stock: 20
Imagen (texto/ruta): jpg
? ¿Disponible? Sí
Producto creado. ID: 6
```

Descripción: Carga de atributos del nuevo producto. El sistema solicita el nombre, el precio unitario, el stock inicial y una descripción detallada, aplicando las validaciones de negocio correspondientes sobre los tipos de datos.

Editar producto:

```
? --- PRODUCTOS --- Editar producto
ID: 3 | Hamburguesa completa | $4500.00 | Stock: 10 | Categoría: Hamburguesas
ID: 6 | hambusguesa mediana | $3550.00 | Stock: 20 | Categoría: Hamburguesas
ID a editar: 6
ID: 2 | Hamburguesas | Hamburguesas y combos
ID categoría [2]: 2
Nombre [hambusguesa mediana]:
Precio [3550.0]: 4000
Descripción [Medallón de carne vacuna, huevo a la plancha, lechuga y tomate en pan brioche.]:
Stock [20]:
Imagen [jpg]:
Disponible S/N [S]:
Producto actualizado.
```

Descripción: se editó el producto (ID: 6). El sistema muestra los datos actuales entre corchetes y permite modificarlos, guardando los cambios al instante.

Eliminar producto:

```
? --- PRODUCTOS --- Eliminar producto
ID: 3 | Hamburguesa completa | $4500.00 | Stock: 10 | Categoría: Hamburguesas
ID: 6 | hambusguesa mediana | $4000.00 | Stock: 20 | Categoría: Hamburguesas
ID a eliminar: 6
? ¿Confirma la eliminación? Sí
Producto eliminado.
```

Descripción: Proceso para borrar un producto. Se ingresa el ID del elemento (ID: 6), el sistema pide una confirmación de seguridad y luego muestra el aviso de que se eliminó con éxito.

USUARIOS:

```
? ¿Qué desea administrar? Usuarios
? --- USUARIOS --- (Usá ↑ ↓ y Enter para seleccionar)
 » Listar usuarios
   Crear usuario
   Editar usuario
   Eliminar usuario
   Volver
```

Descripción: Al seleccionar la opción usuarios, el sistema les mostrara un submenú donde podrán: listar usuarios, crear usuario, editar usuario, eliminar usuario, y la opción de volver que lo llevara al menú principal.

Listar usuarios:

```
? --- USUARIOS --- Listar usuarios
ID: 1 | Ana Perez | admin@foodstore.com | ADMIN
```

Descripción: Listado de los usuarios activos en el sistema. Muestra el ID, el nombre y apellido del usuario, el email y su rol.

Crear usuario:

```
? --- USUARIOS --- Crear usuario
Nombre: fernando
Apellido: pimentel
Mail: ferpim@foodstore.com
Celular: 1154684
Contraseña: fer123
? Seleccione el rol: USUARIO
Usuario creado. ID: 7
```

Descripción: Alta de un nuevo usuario en el sistema. Se completan los datos personales, el correo, el celular, la contraseña y se le asigna el rol de usuario común, generando automáticamente el ID 7.

Editar usuario:

```
? --- USUARIOS --- Editar usuario
ID: 1 | Ana Perez | admin@foodstore.com | ADMIN
ID: 7 | fernando pimentel | ferpim@foodstore.com | USUARIO
ID a editar: 7
Nombre [fernando]:
Apellido [pimentel]:
Mail [ferpim@foodstore.com]:
Celular [1154684]: 1167822
Nueva contraseña (vacío conserva):
? ¿Cambiar rol? No
Usuario actualizado.
```

Descripción: Pantalla para editar un usuario. Se selecciona el ID 7, se actualiza el número de celular manteniendo los demás datos y se confirma que los cambios se guardaron con éxito.

Eliminar usuario:

```
? --- USUARIOS --- Eliminar usuario
ID: 1 | Ana Perez | admin@foodstore.com | ADMIN
ID: 7 | fernando pimentel | ferpim@foodstore.com | USUARIO
ID a eliminar: 7
? ¿Confirma la eliminación? Sí
Usuario eliminado.
```

Descripción: Proceso para borrar un usuario. Se ingresa el ID 7, el sistema pide una confirmación de seguridad y muestra el mensaje de que el usuario fue eliminado.

PEDIDOS:

```
? ¿Qué desea administrar? Pedidos
? --- PEDIDOS --- (Usá ↑ ↓ y Enter para seleccionar)
 » Listar pedidos
   Crear pedido
   Actualizar estado o pago
   Eliminar pedido
   Volver
```

Descripción: Al seleccionar la opción pedidos, el sistema les mostrara un submenú donde podrán: listar pedidos, crear pedido, editar pedido, eliminar pedido, y la opción de volver que lo llevara al menú principal.

Listar pedidos:

```
? --- PEDIDOS --- Listar pedidos
No hay pedidos cargados.
```

Descripción: Listado de pedidos registrados en el sistema. En este caso aun no tenemos pedidos, pero se logra ver que el sistema lo informa correctamente.

Crear pedido:

```
? --- PEDIDOS --- Crear pedido
ID: 1 | Ana Perez | admin@foodstore.com | ADMIN
ID usuario: 1
? Seleccione la forma de pago: EFECTIVO
ID: 3 | Hamburguesa completa | $4500.00 | Stock: 10 | Categoría: Hamburguesas
ID producto: 3
Cantidad: 2
? ¿Agregar otro producto? No
Pedido creado. ID: 8 | Total: $9000.00
```

Descripción: Listado de pedidos registrados en el sistema. En este caso aun no tenemos pedidos, pero se logra ver que el sistema lo informa correctamente.

Actualizar estado o pago:

```
? --- PEDIDOS --- Actualizar estado o pago
ID: 8 | Ana Perez | PENDIENTE | EFECTIVO | $9000.00 | 2026-11-06
ID pedido: 8
? ¿Cambiar estado? Sí
? Seleccione el estado: CONFIRMADO
? ¿Cambiar forma de pago? No
Pedido actualizado.
```

Descripción: se muestra cómo se actualiza un pedido. Se selecciona el ID 8, se cambia el estado de "PENDIENTE" a "CONFIRMADO", se mantiene la forma de pago y el sistema confirma la actualización con éxito.

Eliminar pedido:

```
? --- PEDIDOS --- Eliminar pedido
ID: 8 | Ana Perez | CONFIRMADO | EFECTIVO | $9000.00 | 2026-11-06
ID pedido: 8
? ¿Confirma la eliminación? Sí
Pedido eliminado.
```

Descripción: Proceso para eliminar un pedido. Se ingresa el ID 8, el sistema pide una confirmación de seguridad y muestra el mensaje de que el pedido fue eliminado correctamente.

MENÚ USUARIO:

```
Bienvenido/a, fabricio pimentel [USUARIO]

=== FOOD STORE - CLIENTE (fabricio pimentel) ===
? ¿Qué desea hacer? (Usá ↑ ↓ y Enter para seleccionar)
 » Ver categorías
   Ver productos disponibles
   Mis pedidos
   Mis datos
   Cerrar sesión
   Salir
```

Descripción: Menú principal de la vista de cliente autenticado. Muestra el saludo personalizado con el rol y las opciones de navegación disponibles (ver categorías, productos disponibles, mis pedidos, mis datos, cerrar sesión o salir).

VER CATEGORIAS:

```
? ¿Qué desea hacer? Ver categorías
ID: 2 | Hamburguesas | Hamburguesas y combos
```

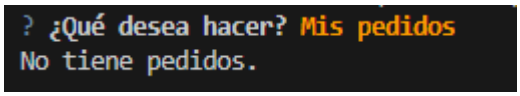
Descripción: Pantalla de visualización de categorías para el cliente. Se listan los rubros activos disponibles en el sistema (ej. ID: 2 - Hamburguesas).

VER PRODUCTOS DISPONIBLES:



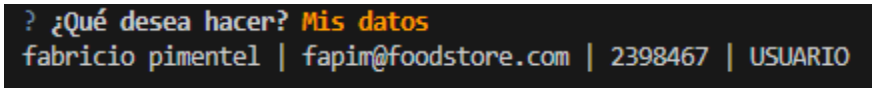
Descripción: Catálogo de productos disponibles para el cliente. Se muestra el detalle de los artículos vigentes con su respectivo precio, stock actual y la categoría a la que pertenecen.

MIS PEDIDOS:



Descripción: Sección de historial de pedidos del cliente. Muestra el estado actual de las compras realizadas por el usuario (en este caso indicando que no posee pedidos registrados).

MIS DATOS:



Descripción: Sección "Mis datos" del cliente. Muestra la información personal registrada de la cuenta activa (nombre y apellido, correo electrónico, número de celular y el rol asignado dentro del sistema).

4. DIFICULTADES Y CONCLUCIONES

Durante el desarrollo del trabajo práctico surgieron algunas dificultades que ayudaron a mejorar la comprensión de los temas vistos.

Consistencia de stock ante cancelaciones.

- Problema: Modificar o cancelar pedidos requería devolver dinámicamente las cantidades al inventario de los productos afectados sin corromper el estado general.
- Solución: Se implementaron restricciones de transiciones de estado en la capa de servicios que ejecutan automáticamente la restitución del stock al cancelar una orden.

Validaciones de unicidad

- Problema: El ingreso de datos como nombres de categorías o correos podía duplicarse variando entre mayúsculas y minúsculas.
- Solución: Normalización de cadenas utilizando métodos nativos de Python (.strip().lower()) antes de realizar las verificaciones en las colecciones.

Conclusión

El desarrollo del proyecto permitió aplicar de forma práctica los fundamentos teóricos de la Programación Orientada a Objetos. La separación en capas, el uso de clases abstractas, interfaces y el polimorfismo demostraron ser

herramientas fundamentales para estructurar un software mantenible, escalable y robusto frente a errores de ejecución.

5. BIBLIOGRAFIA Y WEBGRAFIA

https://www-geeksforgeeks-org.translate.goog/python/abstract-classes-in-python/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc&_x_tr_hist=true

https://ellibrodepython.com/abstract-base-class

https://www.geeksforgeeks.org/python/inheritance-in-python/

https://docs.python.org/es/3.10/library/exceptions.html

https://docs.python.org/es/3/tutorial/errors.html

https://keepcoding.io/blog/como-utilizar-las-type-hints-en-python/

también se utilizó el material de estudio de la carrera.

6. ENLACE VIDEO EXPLICATIVO
https://youtu.be/rhsAGtQr-nM

7. ENLACE REPOSITORIO DE GITHUB
https://github.com/fabriciopimentel87-ship-it/FOOD_STORE.git